

Alumno: Fuentes de la Cruz Bernardo.

Problemario #1: Ciclos y Estructuras en C#.

Instrucciones:

Realiza los siguientes ejercicios como un proyecto independiente de Aplicación de consola en C# dentro de Visual Studio. Para cada ejercicio deberán entregar:

- 1) Análisis del problema: entradas, proceso y salidas.
- 2) Pseudocódigo.
- 3) Diagrama de flujo utilizando la simbología correcta.
- 4) Tipo de variables con las que trabajara el programa.
- 5) Código en C# debidamente indentado y comentado.
- 6) El encabezado en la consola deberá mostrar el planteamiento del problema.
- 7) Evidencia de ejecución en Visual Studio.
- 8) Al menos dos pruebas con datos diferentes.
- 9) Conclusión sobre el funcionamiento del programa.
- 10) Repositorio local Git y GitHub.

Ejercicio 1:

Diseñe un programa que solicite al usuario el voltaje y la corriente consumida por un motor de corriente directa. El programa deberá calcular la potencia eléctrica mediante:

$$P = VI$$

Después deberá clasificar el funcionamiento del motor de acuerdo con lo siguiente:

- a) Si la potencia es menor o igual a 120 W, mostrar: CONSUMO NORMAL.
- b) Si la potencia es mayor que 120 W, mostrar: ADVERTENCIA: CONSUMO ELEVADO.

Problema UI

Ejercicio I

1) Análisis del Problema:

Entradas

- Voltage
- Corriente

Procesos:

- $Potencia = Voltage \times Corriente$
- Si potencia $\leq 120W \rightarrow$ Consumo normal
- Si potencia $> 120W \rightarrow$ Consumo elevado

Salidas:

Valor de la potencia y mensaje de estado

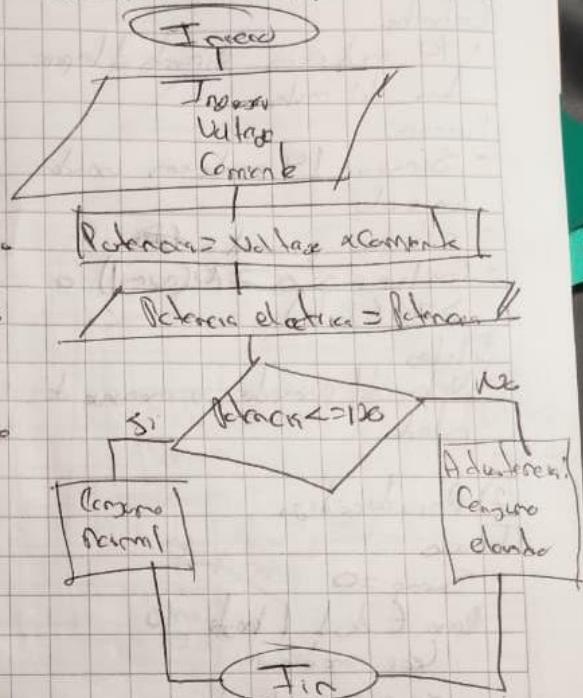
2) Pseudo código:

Inicio

- Pedir Voltage
- Leer voltage
- Pedir Corriente
- Leer corriente
- $Potencia = Voltage \times Corriente$
- Mostrar Potencia
- Si potencia ≤ 120 entonces
 - Consumo normal
- ~~Si potencia > 120 entonces~~
 - ~~Consumo elevado~~

Fin

3) Diagrama de flujo



4) Tipo de variables:

- Voltage $\rightarrow$ double
- Corriente $\rightarrow$ double
- potencia $\rightarrow$ double

Fig. 1: Partes 1), 2), 3) y 4) del ejercicio 1.

5) Código en C#.

```
1 //Diseñe un programa que solicite al usuario el voltaje y la corriente consumida por un motor de corriente directa. El programa deberá calcular la potencia eléctrica.
2 //PsvI
3 //Después deberá clasificar el funcionamiento del motor de acuerdo con lo siguiente:
4 //a) Si la potencia es menor o igual a 120 W, mostrar: CONSUMO NORMAL.
5 //b) Si la potencia es mayor que 120 W, mostrar: ADVERTENCIA: CONSUMO ELEVADO.
6 using System;
7
8 Console.WriteLine("Ejercicio 1"); // Muestra el título del ejercicio en la consola.
9 Console.WriteLine("Este programa calcula la potencia eléctrica de un motor de corriente directa y clasifica su funcionamiento."); // Muestra la descripción del programa
10 Console.WriteLine(); // Imprime un salto de línea en blanco para separar el texto.
11
12 Console.WriteLine("Ingrese el voltaje (V) del motor de corriente directa:"); // Pide al usuario que ingrese el voltaje.
13 double voltaje = Convert.ToDouble(Console.ReadLine()); // Lee el texto ingresado, lo convierte a decimal (double) y lo guarda en la variable 'voltaje'.
14
15 Console.WriteLine("Ingrese la corriente (A) del motor de corriente directa:"); // Pide al usuario que ingrese la corriente.
16 double corriente = Convert.ToDouble(Console.ReadLine()); // Lee el texto ingresado, lo convierte a decimal y lo guarda en la variable 'corriente'.
17 double potencia = voltaje * corriente; // Calcula la potencia multiplicando las dos variables.
18 Console.WriteLine($"La potencia eléctrica del motor es: {potencia} W"); // Imprime en pantalla el resultado de la operación.
19 Console.WriteLine(); // Imprime otro salto de línea.
20
21 if (potencia <= 120) // Evalúa si la potencia es menor o igual a 120 W.
22 {
23     Console.WriteLine("CONSUMO NORMAL."); // Imprime este mensaje si la condición de arriba es verdadera.
24 }
25 else // Entra aquí si la condición fue falsa (potencia mayor a 120 W).
26 {
27     Console.WriteLine("ADVERTENCIA: CONSUMO ELEVADO."); // Imprime este mensaje de alerta.
28 }
```

Fig. 2: Captura del código del ejercicio 1.

6) El encabezado en la consola.

```
C:\Users\berna\OneDrive\Doc x + v
Ejercicio 1
Este programa calcula la potencia eléctrica de un motor de corriente directa y clasifica su funcionamiento.
Ingrese el voltaje (V) del motor de corriente directa:
```

Fig. 3: Captura de la consola mostrando el planteamiento del problema, ejercicio 1.

7) Evidencia de ejecución en Visual Studio & 8) Dos pruebas con datos variados.

```
Consola de depuración de Mi x + v
Ejercicio 1
Este programa calcula la potencia eléctrica de un motor de corriente directa y clasifica su funcionamiento.
Ingrese el voltaje (V) del motor de corriente directa:
20
Ingrese la corriente (A) del motor de corriente directa:
5
La potencia eléctrica del motor es: 100 W
CONSUMO NORMAL.
```

Fig. 4: Ejecución del código con un valor en V=20V y I=5A (Prueba 1).

```
Ejercicio 1
Este programa calcula la potencia eléctrica de un motor de corriente directa y clasifica su funcionamiento.
Ingrese el voltaje (V) del motor de corriente directa:
30
Ingrese la corriente (A) del motor de corriente directa:
9
La potencia eléctrica del motor es: 270 W
ADVERTENCIA: CONSUMO ELEVADO.
```

Fig. 5: Ejecución del código con un valor en V=30V y I=9A (Prueba 2).

9) Conclusión.

El uso de operaciones aritméticas y condicionales (if/else) en C# permite automatizar el cálculo de potencia y diagnosticar el estado operativo de un motor en tiempo real.

Ejercicio 2:

Un sensor mide diez veces la temperatura de un motor. Diseñe un programa en el que se escriba una sola vez las instrucciones para solicitar las diez mediciones, sumarlas y calcular la temperatura promedio. El programa deberá indicar:

- a) TEMPERATURA NORMAL, si el promedio es menor o igual que 70 °C.
- b) ALERTA DE TEMPERATURA, si el promedio es mayor que 70 °C.

Ejercicio 2

1) Análisis del Problema

Entradas

- 10 mediciones diarias de la temperatura del motor

Proceso

- Sumar 10 mediciones corriendo un ciclo
- Calcular promedio $\bar{t}$
- Evaluar si $\bar{t} \leq 70$ (normal) o > 70 (alta)

Salidas

- Valor del promedio y mensaje de estado

2) Pseudocódigo

Inicio

$Suma \leftarrow 0$

Para i desde 1 hasta 10

Leer medición

$Suma \leftarrow Suma + medición$

$Promedio \leftarrow Suma / 10$

Si promedio ≤ 70 entonces

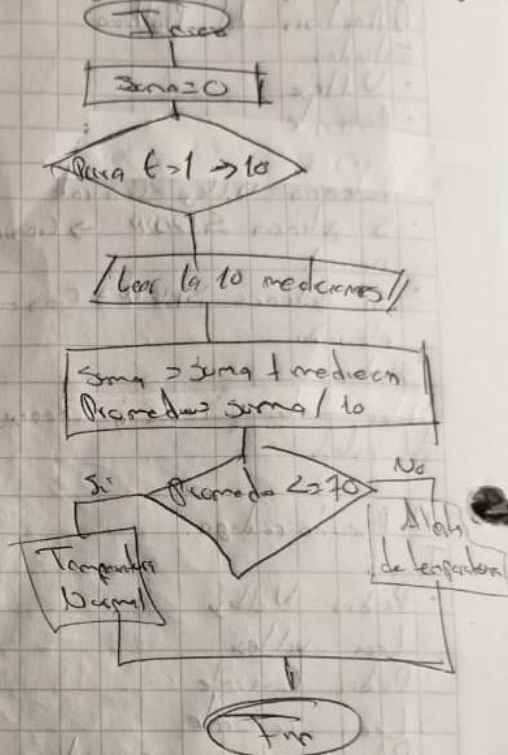
- Temperatura normal

Sino

- Alerta de temperatura

Fin

3) Diagrama de Flujo



4) Tipo de Variables

- $i \rightarrow \text{ent}$
- Suma, medición, promedio $\rightarrow \text{decimales}$

Fig. 6: Partes 1), 2), 3) y 4) del ejercicio 2.

5) Código en C#.

```
1 //Ejercicio 2
2 //Un sensor mide diez veces la temperatura de un motor. Diseñe un programa en el que se escriba una sola vez las
3 //instrucciones para solicitar las diez mediciones, sumarlas y calcular la temperatura promedio. El programa deberá indicar:
4 //a) TEMPERATURA NORMAL, si el promedio es menor o igual que 70 °C.
5 //b) ALERTA DE TEMPERATURA, si el promedio es mayor que 70 °C.
6 using System;
7
8 Console.WriteLine("Ejercicio 2"); // Título.
9 Console.WriteLine("Mediciones de 10 temperaturas de un motor y determine si la temperatura es normal o hay alerta de sobrecalentamiento."); // Descripción.
10 Console.WriteLine(); // Espacio.
11 Console.WriteLine("Ingrese las 10 mediciones de temperatura del motor:"); // Instrucción.
12 double suma = 0; // Acumulador en cero.
13
14 for (int t = 1; t <= 10; t++)
15 {
16     // Se repite 10 veces.
17     Console.Write("Ingrese la mediciones [t]: "); // Pide el dato.
18     double medicion = Convert.ToDouble(Console.ReadLine()); // Lee y convierte a decimal.
19     suma = suma + medicion; // Suma la medición al acumulador.
20 }
21
22 double promedio = suma / 10; // Calcula el promedio.
23 Console.WriteLine("El promedio de las mediciones es: {promedio}"); // Muestra el resultado.
24 Console.WriteLine(); // Espacio.
25
26 if (promedio <= 70)
27 {
28     // Verifica si no supera los 70°C.
29     Console.WriteLine("TEMPERATURA NORMAL"); // Estado normal.
30 }
31 else
32 {
33     Console.WriteLine("ALERTA DE TEMPERATURA"); // Alerta de sobrecalentamiento.
34 }
```

Fig. 7: Captura del código del ejercicio 2.

6) El encabezado en la consola.

```
Ejercicio 2
Mediciones de 10 temperaturas de un motor y determine si la temperatura es normal o hay alerta de sobrecalentamiento.
Ingrese las 10 mediciones de temperatura del motor:
```

Fig. 8: Captura de la consola mostrando el planteamiento del problema, ejercicio 2.

7) Evidencia de ejecución en Visual Studio & 8) Dos pruebas con datos variados.

```
Ejercicio 2
Mediciones de 10 temperaturas de un motor y determine si la temperatura es normal o hay alerta de sobrecalentamiento.
Ingrese las 10 mediciones de temperatura del motor:
Ingrese la mediciones 1: 68
Ingrese la mediciones 2: 64
Ingrese la mediciones 3: 45
Ingrese la mediciones 4: 64
Ingrese la mediciones 5: 13
Ingrese la mediciones 6: 53
Ingrese la mediciones 7: 44
Ingrese la mediciones 8: 33
Ingrese la mediciones 9: 71
Ingrese la mediciones 10: 12
El promedio de las mediciones es: 46.7
TEMPERATURA NORMAL
```

Fig. 9: Resultados de ejecución en consola para el Ejercicio 2 (Prueba 1).

```
Ejercicio 2
Mediciones de 10 temperaturas de un motor y determine si la temperatura es normal o hay alerta de sobrecalentamiento.
Ingrese las 10 mediciones de temperatura del motor:
Ingrese la mediciones 1: 85
Ingrese la mediciones 2: 56
Ingrese la mediciones 3: 87
Ingrese la mediciones 4: 98
Ingrese la mediciones 5: 978
Ingrese la mediciones 6: 86
Ingrese la mediciones 7: 56
Ingrese la mediciones 8: 23
Ingrese la mediciones 9: 2
Ingrese la mediciones 10: 4
El promedio de las mediciones es: 147.5
ALERTA DE TEMPERATURA
```

Fig. 10: Resultados de ejecución en consola para el Ejercicio 2 (Prueba 2).

9) Conclusión.

El uso de ciclos for automatiza la recolección y promedio de múltiples datos, facilitando el monitoreo térmico de motores.

Ejercicio 3:

Diseñe un programa que simule la carga de una batería de 12.6 V. El usuario deberá ingresar:

- a) El voltaje inicial de la batería.
- b) El incremento de voltaje producido en cada ciclo de carga.

Mediante un ciclo while, el programa aumentará el voltaje hasta alcanzar 12.6 V. En cada repetición deberá mostrar el número de ciclo y el voltaje actual.

Si el incremento introducido es igual o menor que cero, el programa deberá mostrar un mensaje de error y no ejecutar la simulación.

Ejercicio 3

1) Análisis del Problema

Entradas

- Voltaje inicial
- Aumento de voltaje por ciclo.

Procesos

- Validar incremento Δ .
- Ciclo while para sumar el incremento al voltaje hasta 12.6V
- Topar límite en 12.6

Salidas

- Mensaje de error (si aplica) o voltaje actual por ciclo y curso de carga completa.

2) Pseudo código.

Inicio

Leer V_{ini}

Leer incremento

Si incremento ≤ 0 entonces
- Error

Síno

ciclo = 0

Mientras $V_{ini} < 12.6$ hacer

ciclo = ciclo + 1

$V_{ini} = V_{ini} + \text{incremento}$

Si $V_{ini} > 12.6$ entonces

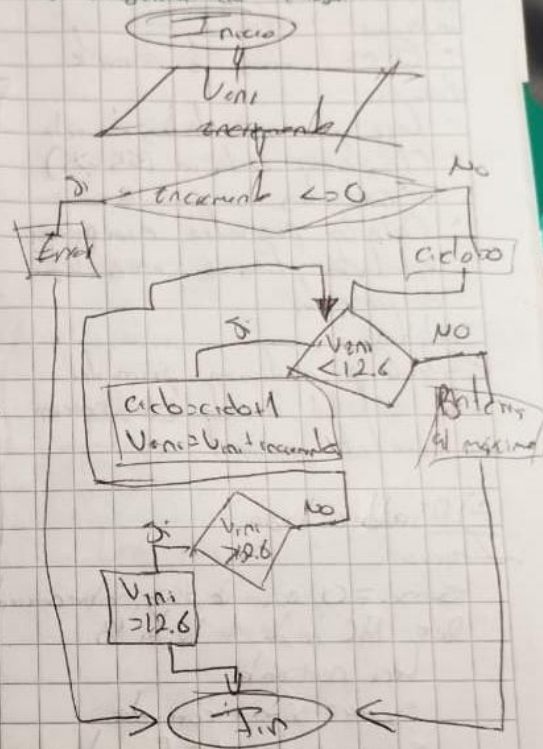
$V_{ini} = 12.6$

Muestra ciclo, voltaje, V_{ini}

Muestra "Batería al máximo"

Fin

3) Diagrama de Flujo



4) Tipos de Variables

- V_{ini} , incremento $\rightarrow$ double
- ciclo $\rightarrow$ int

Fig. 11: Partes 1), 2), 3) y 4) del ejercicio 3.

5) Código en C#.

```
1 //Ejercicio 3
2 //Diseñe un programa que simule la carga de una batería de 12.6 V. El usuario deberá ingresar:
3 //a) el voltaje inicial de la batería.
4 //b) el incremento de voltaje producido en cada ciclo de carga.
5 //Mediante un ciclo while, el programa aumentará el voltaje hasta alcanzar 12.6 V. En cada repetición deberá mostrar el número de ciclo y el voltaje actual.
6 //Si el incremento introducido es igual o menor que cero, el programa deberá mostrar un mensaje de error y no ejecutar la simulación
7 using System;
8 Console.WriteLine("Ejercicio 3"); // Título.
9 Console.WriteLine("Simulación de una batería de 12.6 V"); // Descripción.
10 Console.WriteLine(); // Salto de línea.
11 Console.WriteLine("Ingrese el voltaje inicial de la batería (>=0 hasta 12.6V):"); // Instrucción.
12 double Vini = Convert.ToDouble(Console.ReadLine()); // Lee el voltaje inicial.
13 Console.WriteLine("Ingrese el voltaje producido por cada ciclo de carga (>0):"); // Instrucción.
14 double incremento = Convert.ToDouble(Console.ReadLine()); // Lee el incremento.
15
16 if (incremento <= 0)
17 { // Valida entrada positiva.
18     Console.WriteLine("Error: El incremento debe ser mayor que cero."); // Mensaje de error.
19 }
20 else
21 {
22     int ciclo = 0; // Inicia contador.
23     while (Vini < 12.6) // Repite hasta llegar a 12.6 V.
24     {
25         ciclo++; // Suma 1 al ciclo.
26         Vini += incremento; // Aumenta el voltaje.
27         if (Vini > 12.6) // Verifica límite.
28         {
29             Vini = 12.6; // Restringe al máximo permitido.
30         }
31         Console.WriteLine($"Ciclo {ciclo}: Voltaje actual = {Vini:F2} V"); // Imprime progreso.
32     }
33 }
```

Fig. 12: Captura del código del ejercicio 3.

6) El encabezado en la consola.

```
Ejercicio 3
Simulación de una batería de 12.6 V
Ingrese el voltaje inicial de la batería (>=0 hasta 12.6V):
```

Fig. 13: Captura de la consola mostrando el planteamiento del problema, ejercicio 3.

7) Evidencia de ejecución en Visual Studio & 8) Dos pruebas con datos variados.

```
Ejercicio 3
Simulación de una batería de 12.6 V
Ingrese el voltaje inicial de la batería (>=0 hasta 12.6V):
3
Ingrese el voltaje producido por cada ciclo de carga (>0):
2
Ciclo 1: Voltaje actual = 5.00 V
Ciclo 2: Voltaje actual = 7.00 V
Ciclo 3: Voltaje actual = 9.00 V
Ciclo 4: Voltaje actual = 11.00 V
Ciclo 5: Voltaje actual = 12.60 V
La batería ha alcanzado su voltaje máximo de 12.6 V.
```

Fig. 14: Resultados de ejecución en consola para el Ejercicio 3 (Prueba 1).

```
Ejercicio 3
Simulación de una batería de 12.6 V
Ingrese el voltaje inicial de la batería (>=0 hasta 12.6V):
12
Ingrese el voltaje producido por cada ciclo de carga (>0):
0.1
Ciclo 1: Voltaje actual = 12.10 V
Ciclo 2: Voltaje actual = 12.20 V
Ciclo 3: Voltaje actual = 12.30 V
Ciclo 4: Voltaje actual = 12.40 V
Ciclo 5: Voltaje actual = 12.50 V
Ciclo 6: Voltaje actual = 12.60 V
Ciclo 7: Voltaje actual = 12.60 V
La batería ha alcanzado su voltaje máximo de 12.6 V.
```

Fig. 15: Resultados de ejecución en consola para el Ejercicio 3 (Prueba 2).

9) Conclusión.

El ciclo while es ideal para simular procesos continuos hasta alcanzar un límite específico. Además, validar los datos de entrada y establecer topes máximos garantiza un control seguro y realista del sistema.

Ejercicio 4:

Durante una prueba se realizan ocho mediciones de corriente de un actuador eléctrico. Diseñe un programa que utilice un ciclo para introducir las ocho mediciones. Cada medición deberá clasificarse de la siguiente manera:

- a)** Si la corriente es menor o igual que 5 A, mostrar: MEDICIÓN NORMAL.
- b)** Si es mayor que 5 A, mostrar: SOBRECORRIENTE DETECTADA.

Al finalizar el programa deberá mostrar:

- Corriente promedio.
- Cantidad de mediciones normales.
- Cantidad de sobrecorrientes.
- Resultado general del actuador.

El resultado general será:

- a)** ACTUADOR APROBADO, si no se detectó ninguna sobrecorriente.
- b)** EL ACTUADOR REQUIERE REVISIÓN, si se detectó una o más sobrecorriente

Ejercicio 4

1) Análisis del Problema

Entradas:

- 8 medicinas de cemento.

Procesos:

- Leer y sumar 8 datos en un ciclo.
- Clasificar cada uno (55 o 5)
- Contar los
- Calcular promedio con fórmula y presentar basado en cantidad de cemento.

Salidas:

Gráfico por medicina, promedio total, cantidad de cada clasificación y veredicto general.

2) Pseudocódigo

Inicio

Suma = 0, normales = 0, sobreconcreto = 0

Para MC desde 1 hasta 8

Leer cemento

Suma = suma + cemento

Si cemento < 5 entonces

- Normales

normales++

Sino

Mostrar sobreconcreto

Sobreconcreto++

Fin

Promedio = Suma / 8

Si sobreconcreto > 0 entonces

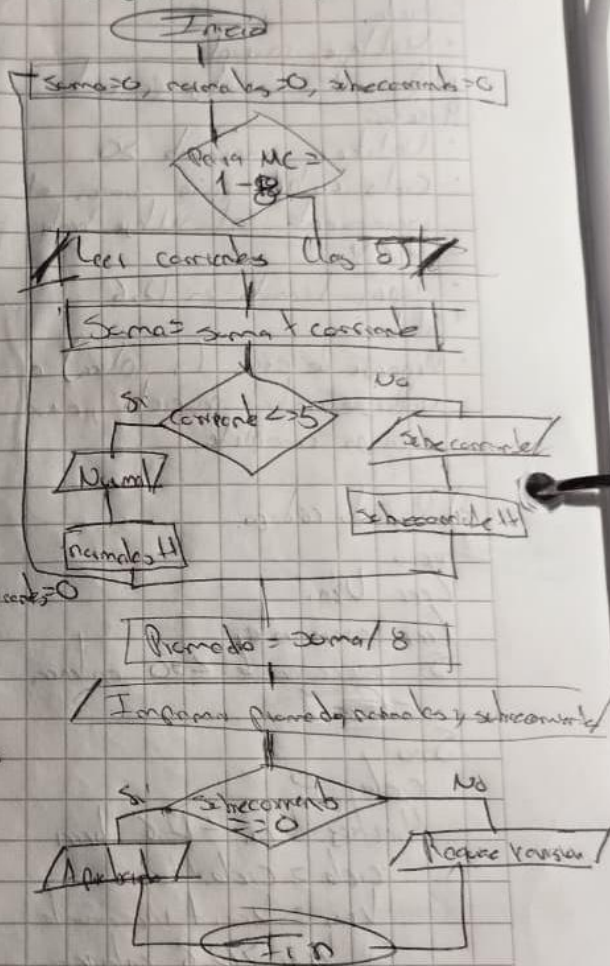
- Mostrar "Aprobado"

Sino

- Mostrar "Requiere revisión"

Fin

3) Diagrama de flujo



4) Tipo de variables

- MC, Cemento, Normales, Sobreconcreto

→ int

- Suma, cemento, promedio → double

Fig. 16: Partes 1), 2), 3) y 4) del ejercicio 4.

5) Código en C#.

```
14 //9) EL ACTUADOR REQUIERE REVISIÓN, SI SE DETECTA UNA O MAS SOBRECORRIENTES
15 using System.Timers;
16
17 Console.WriteLine("Ejercicio 4"); // Título.
18 Console.WriteLine("Simulación de medición de corriente de un actuador eléctrico."); // Descripción.
19 Console.WriteLine(); // Espacio.
20 Console.WriteLine("Ingrese las ocho mediciones de corriente (A):"); // Instrucción.
21 double suma = 0; // Acumulador total.
22 int conteoNormales = 0; // Contador de mediciones normales.
23 int conteoSobrecorrientes = 0; // Contador de fallas.
24
25 for(int MC = 1; MC <= 8; MC++){ // Ciclo de 8 repeticiones.
26     Console.Write($"Medición {MC}: "); // Pide el dato.
27     double corriente = Convert.ToDouble(Console.ReadLine()); // Lee y convierte a decimal.
28     suma += corriente; // Suma la medición al acumulador.
29
30     if(corriente <= 5) // Evalúa si está dentro del límite.
31     {
32         Console.WriteLine("MEDICIÓN NORMAL."); // Mensaje de estado.
33         conteoNormales++; // Incrementa contador normal.
34     }
35     else // Si supera los 5 A.
36     {
37         Console.WriteLine("SOBRECORRIENTE DETECTADA."); // Alerta.
38         conteoSobrecorrientes++; // Incrementa contador de fallas.
39     }
40 }
41
42 double promedio = suma/8; // Calcula el promedio final.
43 Console.WriteLine();
44 Console.WriteLine($"Corriente promedio: {promedio:F2} A"); // Muestra promedio con 2 decimales.
45 Console.WriteLine($"Cantidad de mediciones normales: {conteoNormales}"); // Muestra conteo.
46 Console.WriteLine($"Cantidad de sobrecorrientes: {conteoSobrecorrientes}"); // Muestra conteo.
47 Console.WriteLine();
48
49 if (conteoSobrecorrientes == 0){ // Si no hubo ninguna falla.
50     Console.WriteLine("RESULTADO GENERAL: ACTUADOR APROBADO."); // Dictamen positivo.
51 }
52 else{ // Si hubo al menos una falla.
```

Fig. 17: Captura del código del ejercicio 4.

6) El encabezado en la consola.

```
5
Ejercicio 4
Simulación de medición de corriente de un actuador eléctrico.

Ingrese las ocho mediciones de corriente (A):
```

Fig. 19: Captura de la consola mostrando el planteamiento del problema, ejercicio 4.

7) Evidencia de ejecución en Visual Studio & 8) Dos pruebas con datos variados.

```
using
| Ejercicio 4
| Simulación de medición de corriente de un actuador eléctrico.
Cons
Cons
Cons Ingrese las ocho mediciones de corriente (A):
Cons Medición 1: 5
doub MEDICIÓN NORMAL.
int Medición 2: 4
int MEDICIÓN NORMAL.
for( Medición 3: 3
MEDICIÓN NORMAL.
Medición 4: 2
MEDICIÓN NORMAL.
Medición 5: 1
MEDICIÓN NORMAL.
Medición 6: 2
MEDICIÓN NORMAL.
Medición 7: 3
MEDICIÓN NORMAL.
Medición 8: 5
MEDICIÓN NORMAL.
}
Corriente promedio: 3.12 A
doub Cantidad de mediciones normales: 8
Cons Cantidad de sobrecorrientes: 0
Cons
RESULTADO GENERAL: ACTUADOR APROBADO.
```

Fig. 20: Resultados de ejecución en consola para el Ejercicio 4 (Prueba 1).

```
Ejercicio 4
Simulación de medición de corriente de un actuador eléctrico.

Ingrese las ocho mediciones de corriente (A):
Medición 1: 6
SOBRECORRIENTE DETECTADA.
Medición 2: 5
MEDICIÓN NORMAL.
Medición 3: 8
SOBRECORRIENTE DETECTADA.
Medición 4: 5
MEDICIÓN NORMAL.
Medición 5: 8
SOBRECORRIENTE DETECTADA.
Medición 6: 4
MEDICIÓN NORMAL.
Medición 7: 7
SOBRECORRIENTE DETECTADA.
Medición 8: 1
MEDICIÓN NORMAL.

Corriente promedio: 5.50 A
Cantidad de mediciones normales: 4
Cantidad de sobrecorrientes: 4

RESULTADO GENERAL: EL ACTUADOR REQUIERE REVISIÓN.
```

Fig. 21: Resultados de ejecución en consola para el Ejercicio 4 (Prueba 2).

9) Conclusión.

La integración de ciclos for con contadores condicionales permite automatizar el control de calidad de un actuador, evaluando su historial de fallas para emitir un diagnóstico rápido y preciso.

Ejercicio 5:

Diseñe un programa que permita analizar la vibración de varios motores. Para cada motor, el programa deberá:

- a)** Solicitar el nombre o identificador del motor.
- b)** Pedir cuatro mediciones de vibración mediante un ciclo for.
- c)** Calcular la vibración promedio.
- d)** Clasificar el estado del motor.
- e)** Preguntar si se desea analizar otro motor.

La repetición de motores se controlará mediante un ciclo while. Utilizar la siguiente clasificación:

- Promedio menor o igual que 4.5 mm/s: FUNCIONAMIENTO NORMAL DEL MOTOR.
- Promedio mayor que 4.5 mm/s: EL MOTOR REQUIERE MANTENIMIENTO.

Al finalizar completamente el programa, deberá mostrar:

- a)** Total, de motores evaluados.
- b)** Cantidad de motores con funcionamiento normal.
- c)** Cantidad de motores que requieren mantenimiento.

Ejercicio 5

1) Análisis del Problema

Entradas:

- Identificación del motor
- 4 mediciones de vibración
- Dirección de centrifugación

Proceso:

- Ciclo para múltiples motores
- Ciclo interno para sumar las 4 mediciones
- Calcular promedio
- Clasificar (≤ 4.5 o > 4.5) y de realizar las acciones

Salidas:

Estado y promedio por motor. Tabla de logs (ciclos, minutos, mantenimiento).

2) Pseudocódigo

Inicio

$Tarjetas = 0$, $Minutos = 0$, $Mantenimiento = 0$, $Centrifugación = "S"$

$Mientras centrifugación \geq "S"$ hacer

$Tarjetas = Tarjetas + 1$

Leer motor

$Subvibración = 0$

Para TM desde 1 a 4 hacer

- Leer medición

- $Subvibración = Subvibración + medición$

Fin

$PromVibración = Subvibración / 4$

Si $PromVibración \geq 4.5$ entonces

- Mostrar "Normal"

- $Mantenimiento = Mantenimiento + 1$

Fin

- Escalar "Mantenimiento"

- $Mantenimiento = Mantenimiento + 1$

Fin

Leer centrifugación

Escalar $Tarjetas$, $Minutos$, $Mantenimiento$.

Fin

2) Diagrama de Flujo

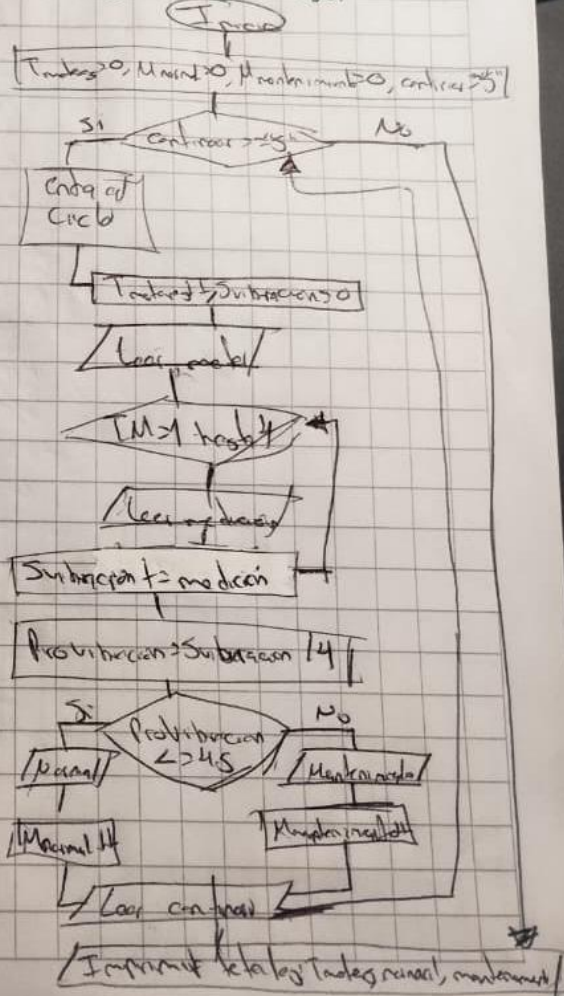


Fig. 22: Partes 1), 2) y 3) del ejercicio 5.

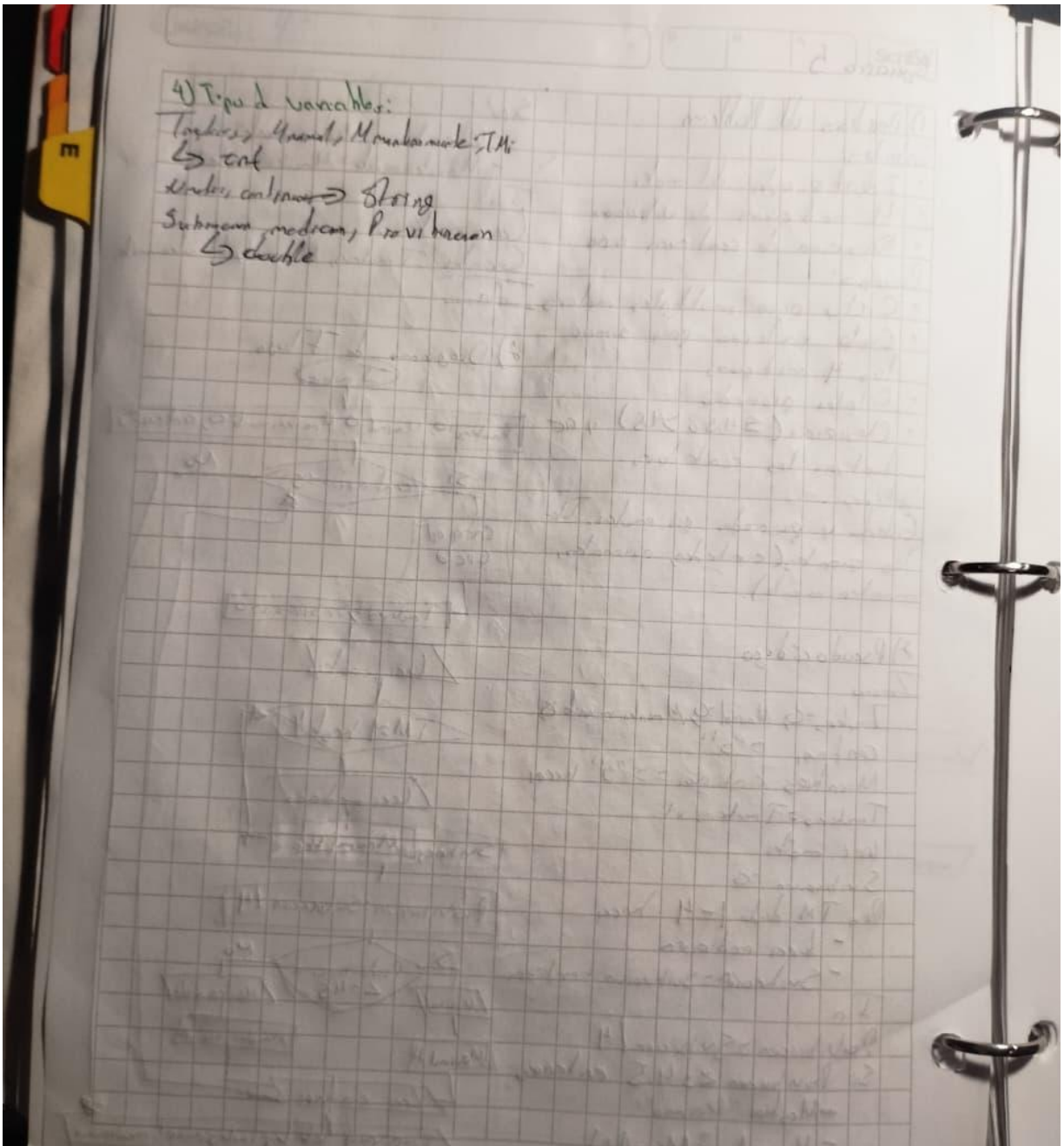


Fig. 23: Parte 4) del ejercicio 5.

5) Código en C#.

```
15 using System;
16 Console.WriteLine("Ejercicio 5"); // Título.
17 Console.WriteLine("Simulación de medición de vibración de varios motores."); // Descripción.
18 Console.WriteLine(); // Espacio.
19 int Tmotores = 0; // Contador global de motores evaluados.
20 int Mnormal = 0; // Contador de motores en buen estado.
21 int Mmantenimiento = 0; // Contador de motores que requieren revisión.
22 string continuar = "s"; // Variable de control del ciclo principal.
23
24 while (continuar == "s" || continuar == "S")
25 { // Repite mientras el usuario decida continuar.
26     Tmotores++; // Incrementa el número del motor actual.
27     Console.WriteLine($"Motor {Tmotores}.");
28     Console.Write("Ingrese el nombre o identificador del motor: "); // Pide ID.
29     string Nmotor = Console.ReadLine(); // Lee el ID ingresado.
30     double Svibracion = 0; // Acumulador local (se reinicia a 0 para cada motor).
31
32     for (int TM = 1; TM <= 4; TM++) // Ciclo interno para 4 mediciones exactas.
33     {
34         Console.Write($"Ingrese la medición de vibración {TM} (mm/s): "); // Pide medición.
35         double medicion = Convert.ToDouble(Console.ReadLine()); // Lee y convierte a decimal.
36         Svibracion += medicion; // Suma la lectura al acumulador local.
37     }
38
39     double ProVibracion = Svibracion / 4; // Calcula el promedio del motor actual.
40     Console.WriteLine($"Promedio de vibración del motor {Nmotor}: {ProVibracion:F2} mm/s"); // Imprime resultado con 2 decimales.
41
42     if (ProVibracion <= 4.5) // Evalúa si la vibración está dentro del límite.
43     {
44         Console.WriteLine("FUNCIONAMIENTO NORMAL DEL MOTOR."); // Dictamen positivo.
45         Mnormal++; // Incrementa el contador global de motores normales.
46     }
47     else // Si supera el límite de 4.5.
48     {
49         Console.WriteLine("EL MOTOR REQUIERE MANTENIMIENTO."); // Dictamen negativo.
50         Mmantenimiento++; // Incrementa el contador global de fallas.
51     }
52     Console.WriteLine("¿Continúa? (s/n): "); // Pide si continúa.
53     continuar = Console.ReadLine(); // Lee la respuesta.
54 }
```

Fig. 24: Captura del código del ejercicio 5.

6) El encabezado en la consola.

```
1 Ejercicio 5
2 Simulación de medición de vibración de varios motores.
```

Fig. 25: Captura de la consola mostrando el planteamiento del problema, ejercicio 5.

7) Evidencia de ejecución en Visual Studio & 8) Dos pruebas con datos variados.

```
Ejercicio 5
Simulación de medición de vibración de varios motores.

Motor 1.
Ingrese el nombre o identificador del motor: alpha
Ingrese la medición de vibración 1 (mm/s): 3
Ingrese la medición de vibración 2 (mm/s): 4
Ingrese la medición de vibración 3 (mm/s): 2
Ingrese la medición de vibración 4 (mm/s): 5
Promedio de vibración del motor alpha: 3.50 mm/s
FUNCIONAMIENTO NORMAL DEL MOTOR.
¿Desea analizar otro motor? (s/n): s
Motor 2.
Ingrese el nombre o identificador del motor: beta
Ingrese la medición de vibración 1 (mm/s): 6
Ingrese la medición de vibración 2 (mm/s): 4
Ingrese la medición de vibración 3 (mm/s): 5
Ingrese la medición de vibración 4 (mm/s): 6
Promedio de vibración del motor beta: 5.25 mm/s
EL MOTOR REQUIERE MANTENIMIENTO.
¿Desea analizar otro motor? (s/n): s
Motor 3.
Ingrese el nombre o identificador del motor: 6
Ingrese la medición de vibración 1 (mm/s): 4
Ingrese la medición de vibración 2 (mm/s): 2
Ingrese la medición de vibración 3 (mm/s): 3
Ingrese la medición de vibración 4 (mm/s): 1
Promedio de vibración del motor 6: 2.50 mm/s
FUNCIONAMIENTO NORMAL DEL MOTOR.
¿Desea analizar otro motor? (s/n): n
Total de motores evaluados: 3
```

Fig. 26: Resultados de ejecución en consola para el Ejercicio 5 (Prueba 1).

```
Ejercicio 5
Simulación de medición de vibración de varios motores.

Motor 1.
Ingrese el nombre o identificador del motor: god
Ingrese la medición de vibración 1 (mm/s): 7
Ingrese la medición de vibración 2 (mm/s): 6
Ingrese la medición de vibración 3 (mm/s): 7
Ingrese la medición de vibración 4 (mm/s): 8
Promedio de vibración del motor god: 7.00 mm/s
EL MOTOR REQUIERE MANTENIMIENTO.
¿Desea analizar otro motor? (s/n): s
Motor 2.
Ingrese el nombre o identificador del motor: bless
Ingrese la medición de vibración 1 (mm/s): 2
Ingrese la medición de vibración 2 (mm/s): 3
Ingrese la medición de vibración 3 (mm/s): 4
Ingrese la medición de vibración 4 (mm/s): 1
Promedio de vibración del motor bless: 2.50 mm/s
FUNCIONAMIENTO NORMAL DEL MOTOR.
¿Desea analizar otro motor? (s/n): n
Total de motores evaluados: 2
Motores con funcionamiento normal: 1
Motores que requieren mantenimiento: 1
```

Fig. 27: Resultados de ejecución en consola para el Ejercicio 5 (Prueba 2).

9) Conclusión.

La combinación de ciclos anidados (while y for) permite procesar múltiples mediciones por equipo de forma continua, facilitando diagnósticos individuales y generando un reporte global automatizado.